ОТЧЁТ

по лабораторной работе № 4

«Линейная алгебра»

Базлов Владимир

Цель работы

Освоить основные операции линейной алгебры в Julia: скалярное и внешнее произведения векторов, решение систем линейных уравнений, диагонализацию матриц, вычисление степеней и квадратных корней матриц, а также исследование продуктивности линейных экономических моделей.

Задание

Выполнить задания раздела 4.4 лабораторной работы: 4.4.1 — произведения векторов; 4.4.2 — системы линейных уравнений; 4.4.3 — операции с матрицами и собственными значениями; 4.4.4 — линейные экономические модели. Код запускать в JupyterLab с ядром Julia, фиксируя результаты скриншотами.

Выполнение работы

4.4.1. Скалярное и внешнее произведения вектора

Создадим вектор v. Скалярное произведение вектора на себя вычисляется функцией dot, а внешнее произведение — произведением вектора-столбца на вектор-строку.

```
v = [1, 2, 3]
dot_v = dot(v, v)
outer_v = v * v'
println("Скалярное произведение: ", dot_v)
println("Внешнее произведение:")
display(outer_v)
```

Для выбранного вектора скалярное произведение равно $1^2 + 2^2 + 3^2 = 14$. Внешнее произведение представляет собой матрицу 3×3.

```
# Ожидаемый результат:
# Скалярное произведение: 14
# 3×3 матрица:
# 1  2  3
# 2  4  6
# 3  6  9
```

[162]:
```
using LinearAlgebra

v = [1, 2, 3]

dot_v = dot(v,v)

println("dot_v = ", dot_v)
```

```
dot_v = 14
```

4.4.2. Решение систем линейных уравнений

Системы записываются в матричном виде A*x = b. Для совместной системы Julia находит решение оператором A \ b. Если у системы бесконечно много решений или она несовместна, это определяется по рангу матрицы коэффициентов и расширенной матрицы.

4.4.2.1. Системы с двумя неизвестными

Для каждой системы задаются матрица коэффициентов A и вектор правой части b. В первых четырёх примерах система имеет зависимые уравнения либо несовместные условия; в остальных примерах проверяется совместность.

```
using LinearAlgebra

systems2 = [
    ([1 1; 1 -1], [2, 3]),   # a
    ([1 1; 2 2], [2, 4]),    # b
    ([1 1; 2 2], [2, 5]),    # c
    ([1 1; 2 2; 3 3], [1, 2, 3]), # d
    ([1 1; 2 1; 1 -1], [2, 1, 3]), # e
    ([1 1; 2 1; 3 2], [2, 1, 3])   # f
]

for (i, (A, b)) in enumerate(systems2)
    println("Система ", ('a':'f')[i])
    println("rank(A) = ", rank(A), ", rank([A b]) = ", rank(hcat(A, b)))
    if rank(A) == rank(hcat(A, b))
        if rank(A) == size(A, 2)
            println("Единственное решение: ", A \ b)
        else
            println("Бесконечно много решений")
        end
    else
        println("Решений нет")
    end
end
```

Результаты анализа:

а) единственное решение: x = 2,5; y = −0,5;

```
[168]: A_a = [1 1; 1 -1]
       b_a = [2,3]
       x_a = A_a \ b_a
       println(x_a)

       [2.5, -0.5]
```

б) бесконечно много решений: x + y = 2;

[170]:
```
A_b = [1 1; 2 2]
b_b = [2, 4]
println("rank(A) = ", rank(A_b))
println("rank[A b]) = ", rank(hcat(A_b, b_b)))
```

```
rank(A) = 1
rank[A b]) = 1
```

в) решений нет;

[172]:
```
A_c =[1 1; 2 2]
b_c = [2, 5]
println("rank(A) = ", rank(A_c))
println("rank[A b]) = ", rank(hcat(A_c, b_c)))
```

```
rank(A) = 1
rank[A b]) = 2
```

г) бесконечно много решений: x + y = 1;

[174]:
```
A_d = [1 1; 2 2; 3 3]
b_d = [1, 2, 3]
println("rank(A) = ", rank(A_d))
println("rank[A b]) = ", rank(hcat(A_d, b_d)))
```

```
rank(A) = 1
rank[A b]) = 1
```

д) решений нет;

[176]:
```
A_e = [1 1; 2 1; 1 -1]
b_e = [2, 1, 3]
println("rank(A) = ", rank(A_e))
println("rank[A b]) = ", rank(hcat(A_e, b_e)))
```

```
rank(A) = 2
rank[A b]) = 3
```

е) единственное решение: x = −1; y = 3.

[178]:
```
A_f = [1 1; 2 1; 3 2]
b_f = [2, 1,3]
x_f = A_f \ b_f
println(x_f)
```

```
[-0.999999999999999, 2.9999999999999982]
```

4.4.2.2. Системы с тремя неизвестными

Аналогично исследуем системы с неизвестными x, y, z. В системе (a) два уравнения и три неизвестных, поэтому при совместности решений бесконечно много.

```
systems3 = [
```

```
    ([1 1 1; 1 -1 -2], [2, 3]),        # a
    ([1 1 1; 2 2 -3; 3 1 1], [2, 4, 1]), # b
    ([1 1 1; 1 1 2; 2 2 3], [1, 0, 1]),  # c
    ([1 1 1; 1 1 2; 2 2 3], [1, 0, 0])   # d
]

for (i, (A, b)) in enumerate(systems3)
    println("Система ", ('a':'d')[i])
    println("rank(A) = ", rank(A), ", rank([A b]) = ", rank(hcat(A, b)))
    if rank(A) == rank(hcat(A, b))
        println(rank(A) == size(A, 2) ? "Единственное решение" : "Бесконечно много решений")
        if rank(A) == size(A, 2)
            println(A \ b)
        end
    else
        println("Решений нет")
    end
end
```

Результаты:

а) бесконечно много решений: при z = t получаем x = (5 + t)/2, y = (−1 − 3t)/2;

[180]:
```
A3a = [1 1 1; 1 -1 -2]
b3a = [2, 3]
println("rank(A) = ", rank(A3a))
println("rank[A b]) = ", rank(hcat(A3a, b3a)))
```

```
rank(A) = 2
rank[A b]) = 2
```

б) единственное решение: x = −0,5; y = 2,5; z = 0;

[182]:
```
A3b = [1 1 1; 2 2 -3; 3 1 1]
b3b = [2, 4, 1]
x3b = A3b \b3b
println(x3b)
```

```
[-0.5, 2.5, 0.0]
```

в) бесконечно много решений: x + y = 2, z = −1;

[184]:
```
A3c = [1 1 1; 1 1 2; 2 2 3]
b3c = [1, 0, 1]
println("rank(A) = ", rank(A3c))
println("rank[A b]) = ", rank(hcat(A3c, b3c)))
```

```
rank(A) = 2
rank[A b]) = 2
```

г) решений нет.

[186]:
```
A3d = [1 1 1; 1 1 2; 2 2 3]
b3d = [1, 0, 0]
println("rank(A) = ", rank(A3d))
println("rank[A b]) = ", rank(hcat(A3d, b3d)))
```

```
rank(A) = 2
rank[A b]) = 3
```

4.4.3. Операции с матрицами

4.4.3.1. Диагонализация матриц

Для симметричных матриц выполняется спектральное разложение A = Q*D*Q', где столбцы Q — ортонормированные собственные векторы, а D — диагональная матрица собственных значений.

```
matrices = [
    [1.0 -2.0; -2.0 1.0],
    [1.0 -2.0; -2.0 3.0],
    [1.0 -2.0 0.0; -2.0 1.0 2.0; 0.0 2.0 0.0]
]

for (i, A) in enumerate(matrices)
    F = eigen(Symmetric(A))
    println("Матрица ", ('a':'c')[i])
    println("Собственные значения: ", F.values)
    println("Диагональная матрица D:")
    display(Diagonal(F.values))
    println("Матрица собственных векторов Q:")
    display(F.vectors)
    println("Проверка A ≈ Q*D*Q': ", isapprox(A, F.vectors * Diagonal(F.values) * F.vectors'))
end
```

Для каждой матрицы выводятся собственные значения, диагональная матрица D и матрица собственных векторов Q. Последняя проверка должна вернуть true с учётом вычислительной погрешности.

[188]:
```
using LinearAlgebra
A = [1.0 -2; -2 1]
E = eigen(A)
D = Diagonal(E.values)
V = E.vectors
display(D)
display(V)
display(V * D * inv(V))
```

```
2×2 Diagonal{Float64, Vector{Float64}}:
 -1.0    ⋅
  ⋅     3.0
2×2 Matrix{Float64}:
 -0.707107  -0.707107
 -0.707107   0.707107
2×2 Matrix{Float64}:
  1.0  -2.0
 -2.0   1.0
```

[190]:
```
A = [1.0 -2; -2 3]
E = eigen(A)
D = Diagonal(E.values)
V = E.vectors
display(E.values)
display(D)
display(V * D * inv(V))
```

```
2-element Vector{Float64}:
 -0.2360679774997897
  4.23606797749979
2×2 Diagonal{Float64, Vector{Float64}}:
 -0.236068    ⋅
   ⋅         4.23607
2×2 Matrix{Float64}:
  1.0  -2.0
 -2.0   3.0
```

[192]:
```
A = [1.0 -2 0; -2 1 2; 0 2 0]
E = eigen(A)
D = Diagonal(E.values)
V = E.vectors
display(E.values)
display(D)
display(V * D * inv(V))
```

```
3-element Vector{Float64}:
 -2.1413361156553594
  0.5151380471280724
  3.6261980685272936
3×3 Diagonal{Float64, Vector{Float64}}:
 -2.14134    ⋅         ⋅
   ⋅        0.515138   ⋅
   ⋅         ⋅        3.6262
3×3 Matrix{Float64}:
  1.0          -2.0  -1.11022e-16
 -2.0           1.0   2.0
 -2.22045e-16   2.0   5.88418e-15
```

4.4.3.2. Степень и квадратные корни матриц

Вычислим десятую степень матрицы из пункта (а) и проверим существование вещественных квадратных корней матриц из пунктов (б–г). Для симметричной матрицы положительный квадратный корень можно построить по спектральному разложению, если все собственные значения неотрицательны.

```
A_a = [1.0 -2.0; -2.0 1.0]
println("A_a^10 =")
display(A_a^10)

function symmetric_sqrt(A)
    F = eigen(Symmetric(A))
    minimum(F.values) < -1e-10 && error("Вещественного симметричного квадратного корня нет:
есть отрицательное собственное значение")
    return F.vectors * Diagonal(sqrt.(max.(F.values, 0.0))) * F.vectors'
```

```
end

A_b = [5.0 -2.0; -2.0 5.0]
A_c = [1.0 -2.0; -2.0 1.0]
A_d = [1.0 2.0; 2.0 3.0]

for (label, A) in [("b", A_b), ("c", A_c), ("d", A_d)]
    println("Матрица ", label)
    try
        R = symmetric_sqrt(A)
        display(R)
        println("Проверка R^2 ≈ A: ", isapprox(R^2, A))
    catch e
        println(e)
    end
end
```

Для (а) результат A^10 = [29525 −29524; −29524 29525]. У матрицы (б) собственные значения положительны, поэтому вещественный симметричный квадратный корень существует. Для (в) и (г) имеются отрицательные собственные значения, поэтому вещественного квадратного корня в общем случае для этих матриц нет.

[194]:
```
A = [1 -2; -2 1]
result_a = A^10
display(result_a)
```

```
2×2 Matrix{Int64}:
  29525  -29524
 -29524   29525
```

[200]:
```
B = [5.0 -2; -2 5]
E = eigen(B)
S = E.vectors * Diagonal(sqrt.(E.values)) * E.vectors'
display(S)
display(S * S)
```

```
2×2 Matrix{Float64}:
  2.1889   -0.45685
 -0.45685   2.1889
2×2 Matrix{Float64}:
  5.0  -2.0
 -2.0   5.0
```

[202]:
```
C = [1.0 -2; -2 1]
E = eigen(C)
display(E.values)
if all(E.values .>= 0)
    S = E.vectors * Diagonal(sqrt.(E.values)) * E.vectors'
    display(S)
else
    println("корень не существует")
end
```

```
2-element Vector{Float64}:
 -1.0
  3.0
корень не существует
```

[204]:
```
D = [1.0 2; 2 3]
E = eigen(D)
display(E.values)
if all(E.values .>= 0)
    S = E.vectors * Diagonal(sqrt.(E.values)) * E.vectors'
    display(S)
else
    println("корень не существует")
end
```

```
2-element Vector{Float64}:
 -0.2360679774997897
  4.23606797749979
корень не существует
```

4.4.3.3. Собственные значения и треугольная матрица

Для заданной матрицы 5×5 найдём собственные значения, диагональную матрицу собственных значений и нижнюю треугольную часть исходной матрицы. Для измерения времени используется макрос @btime из пакета BenchmarkTools.

```
using LinearAlgebra
using Pkg
# Выполнить один раз, если пакет ещё не установлен:
# Pkg.add("BenchmarkTools")
using BenchmarkTools

A5 = [140 97 74 168 131;
       97 106 89 131 36;
       74 89 152 144 71;
      168 131 144 54 142;
      131 36 71 142 36]

values = eigvals(Symmetric(A5))
D = Diagonal(values)
L = tril(A5)

println("Собственные значения: ", values)
println("Диагональная матрица:")
display(D)
```

```
println("Нижняя треугольная матрица:")
display(L)

println("Время вычисления собственных значений:")
@btime eigvals(Symmetric($A5))
println("Время построения нижней треугольной матрицы:")
@btime tril($A5)
```

Матрица A5 симметрична, поэтому применяется eigvals(Symmetric(A5)). Функция tril оставляет диагональ и элементы ниже диагонали. @btime показывает оценку времени выполнения операции; значения времени зависят от компьютера и окружения.

[206]:
```
using LinearAlgebra
using BenchmarkTools

A5 = [
    140  97  74 168 131;
     97 106  89 131  36;
     74  89 152 144  71;
    168 131 144  54 142;
    131  36  71 142  36
]

# Собственные значения
values = eigvals(Symmetric(A5))

# Диагональная матрица собственных значений
D5 = Diagonal(values)

# Нижнетреугольная матрица
L5 = tril(A5)

display(values)
display(D5)
display(L5)

# Оценка времени вычислений
@btime eigvals(Symmetric($A5))
@btime tril($A5)
```

```
5-element Vector{Float64}:
 -128.49322764802145
  -55.88778455305688
   42.75216727931894
   87.16111477514521
  542.4677301466143
5×5 Diagonal{Float64, Vector{Float64}}:
 -128.493      ⋅         ⋅         ⋅         ⋅
     ⋅      -55.8878     ⋅         ⋅         ⋅
     ⋅         ⋅      42.7522     ⋅         ⋅
     ⋅         ⋅         ⋅      87.1611      ⋅
     ⋅         ⋅         ⋅         ⋅      542.468
5×5 Matrix{Int64}:
 140    0    0    0   0
  97  106    0    0   0
  74   89  152    0   0
 168  131  144   54   0
 131   36   71  142  36
  1.870 μs (13 allocations: 2.44 KiB)
  46.162 ns (2 allocations: 272 bytes)
```

4.4.4. Линейные экономические модели

Рассматривается модель x − A*x = y, где A — матрица прямых затрат, x — вектор валового выпуска, y — вектор конечного спроса. При обратимости I − A решение имеет вид x = (I −

A)$^{-1}$ у. В стандартном критерии продуктивности проверяют неотрицательность матрицы (I − A)$^{-1}$; спектральный критерий требует, чтобы спектральный радиус A был меньше единицы.

Примечание: матрицы и множители ниже перенесены по распознанному тексту методички; перед сдачей сверьте их с исходным PDF, поскольку в скане дроби могут отображаться неразборчиво.

4.4.4.1. Проверка продуктивности по определению

Для каждого варианта вычисляется матрица B = (I − A)$^{-1}$. Проверяем, что все её элементы неотрицательны.

```
M = [1.0 2.0; 3.0 4.0]
As = [M, 0.5 .* M, 0.1 .* M]

for (i, A) in enumerate(As)
    println("Вариант ", ('a':'c')[i])
    B = inv(Matrix{Float64}(I, size(A, 1), size(A, 2)) - A)
    display(B)
    println("Все элементы обратной матрицы неотрицательны: ", all(B .>= -1e-10))
end
```

[208]:
```
using LinearAlgebra

M = [1.0 2; 3 4]

A_a = M
A_b = (1/2) .* M
A_c = (1/10) .* M

function check_by_definition(A)
    B = I - A

    if abs(det(B)) < 1e-10
        println("I - A вырождена: единственного решения нет")
        return
    end

    C = inv(B)
    println("Матрица A:")
    display(A)
    println("(I - A)^(-1):")
    display(C)
    println("Продуктивна: ", all(C .>= -1e-10))
end

check_by_definition(A_a)
check_by_definition(A_b)
check_by_definition(A_c)
```

```
Матрица A:
2×2 Matrix{Float64}:
 1.0  2.0
 3.0  4.0
(I - A)^(-1):
2×2 Matrix{Float64}:
  0.5  -0.333333
 -0.5   0.0
Продуктивна: false
Матрица A:
2×2 Matrix{Float64}:
 0.5  1.0
 1.5  2.0
(I - A)^(-1):
2×2 Matrix{Float64}:
  0.5   -0.5
 -0.75  -0.25
Продуктивна: false
Матрица A:
2×2 Matrix{Float64}:
 0.1  0.2
 0.3  0.4
(I - A)^(-1):
```

4.4.4.2. Критерий обратимости

Для следующего набора матриц проверяем обратимость $I - A$. Если определитель равен нулю (или матрица численно вырождена), обратной матрицы не существует.

```
M2 = [1.0 2.0; 3.0 1.0]
As2 = [M2, 0.5 .* M2, 0.1 .* M2]

for (i, A) in enumerate(As2)
    B = Matrix{Float64}(I, size(A, 1), size(A, 2)) - A
    println("Вариант ", ('a':'c')[i])
    println("det(I-A) = ", det(B))
    println("Обратима ли I-A: ", abs(det(B)) > 1e-10)
end
```

Обратимость $I - A$ является необходимым условием единственности решения модели для произвольного вектора спроса y. Сама по себе обратимость ещё не гарантирует продуктивность — дополнительно проверяется знак обратной матрицы или спектральный радиус.

[210]:
```
M = [1.0 2; 3 1]

A_a = M
A_b = (1/2) .* M
A_c = (1/10) .* M

function check_inverse_criterion(A)
    B = I - A

    println("A = ")
    display(A)

    if abs(det(B)) < 1e-10
        println("Обратная матрица не существует")
        return
    end

    C = inv(B)
    display(C)

    println("Продуктивна: ", all(C .>= -1e-10))
end

check_inverse_criterion(A_a)
check_inverse_criterion(A_b)
check_inverse_criterion(A_c)
```

```
A =
2×2 Matrix{Float64}:
 1.0  2.0
 3.0  1.0
2×2 Matrix{Float64}:
 -0.0  -0.333333
 -0.5   0.0
Продуктивна: false
A =
2×2 Matrix{Float64}:
 0.5  1.0
 1.5  0.5
2×2 Matrix{Float64}:
 -0.4  -0.8
 -1.2  -0.4
Продуктивна: false
A =
2×2 Matrix{Float64}:
```

4.4.4.3. Спектральный критерий продуктивности

Вычисляем спектральный радиус ρ(A) — максимальный модуль собственных значений. Модель удовлетворяет спектральному критерию продуктивности, если ρ(A) < 1.

```
A_a = [1.0 2.0; 3.0 1.0]
A_b = 0.5 .* A_a
A_c = 0.1 .* A_a
A_d = [0.1 0.2 0.3;
       0.0 0.1 0.2;
       0.0 0.1 0.3]

for (label, A) in [("a", A_a), ("b", A_b), ("c", A_c), ("d", A_d)]
    rho = maximum(abs.(eigvals(A)))
    println("Вариант ", label, ": спектральный радиус = ", rho,
            "; продуктивна = ", rho < 1)
end
```

По критерию ρ(A) < 1 варианты (а) и (б) для матрицы M2 не являются продуктивными, вариант (в) продуктивен; для трёхмерной матрицы (г) спектральный радиус равен 0,3, поэтому она продуктивна.

[212]:
```
function check_spectral_criterion(A)
    values = eigvals(A)

    println("Собственные значения:")
    display(values)

    println("Модули собственных значений:")
    display(abs.(values))

    println("Продуктивна: ", all(abs.(values) .< 1))
end

M = [1.0 2; 3 1]

check_spectral_criterion(M)
check_spectral_criterion((1/2) .* M)
check_spectral_criterion((1/10) .* M)

A_d = [
    0.1 0.2 0.3;
    0.0 0.1 0.2;
    0.0 0.1 0.3
]

check_spectral_criterion(A_d)
```

```
Собственные значения:
2-element Vector{Float64}:
 -1.4494897427831779
  3.4494897427831783
Модули собственных значений:
2-element Vector{Float64}:
 1.4494897427831779
 3.4494897427831783
Продуктивна: false
Собственные значения:
2-element Vector{Float64}:
 -0.7247448713915892
  1.724744871391589
Модули собственных значений:
2-element Vector{Float64}:
 0.7247448713915892
 1.724744871391589
Продуктивна: false
Собственные значения:
2-element Vector{Float64}:
 -0.14494897427831785
  0.34494897427831783
Модули собственных значений:
2-element Vector{Float64}:
 0.14494897427831785
```

Выводы

В ходе лабораторной работы были выполнены основные операции линейной алгебры в Julia. Вычислены скалярное и внешнее произведения вектора; системы линейных уравнений исследованы с помощью рангов матрицы коэффициентов и расширенной матрицы; для симметричных матриц получены собственные значения и диагональные представления; выполнены возведение матрицы в степень, проверка квадратных корней и анализ матрицы

5×5. В заключительной части рассмотрена модель x − A*x = y и изучены критерии продуктивности: неотрицательность $(I - A)^{-1}$ и условие $\rho(A) < 1$.

Перед сдачей необходимо вставить скриншоты в отмеченные места, проверить соответствие матриц пунктам 4.4.4 исходной методички и убедиться, что все ячейки выполняются без ошибок.